

EECS 3311 Software Design
Deliverable 1

Group Members:

Trinity Mae Robel	220532479
Ava Yunus	218873471
Abel Sabu	219139476
Haider Malik	220400297
Aleena Aleem	219027945
Christian Rosel	218748939

Part I: Requirements Eliciting and Modeling - 50 points

Task 1: Use Case Diagram - 20 points (**Haider Malik**)

Task 2: Activity Diagram - 15 points (**Trinity Mae Robel**)

Task 3: Sequence Diagram - 15 points (**Christian Rosel**)

Part II: Design - 20 points

Task 1: Class Diagram - 20 points (**Aleena Aleem**)

Part III: Report on AI-Assistant Adaption - 30 points

Task 1: Explain the Role of the AI Assistant (**Abel Sabu**)

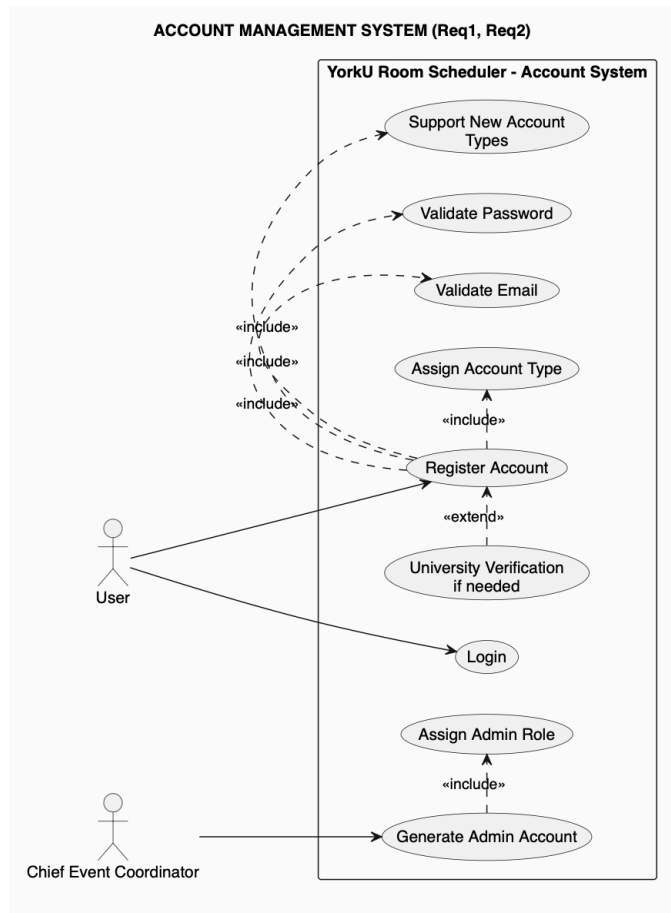
Task 2: Arguments and Resolutions (**Ava Yunus**)

Task 3: Verifying AI Responses (**Ava Yunus**)

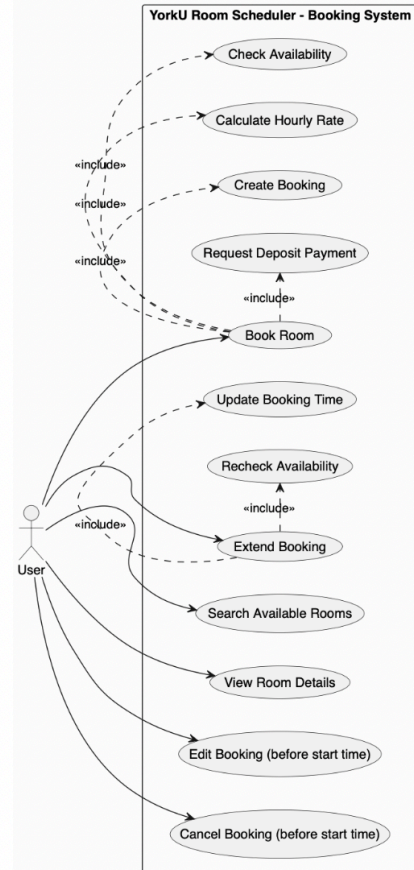
Task 4: Application of AI-Suggested Content (**Ava Yunus**)

Part I: Requirements Eliciting and Modelling

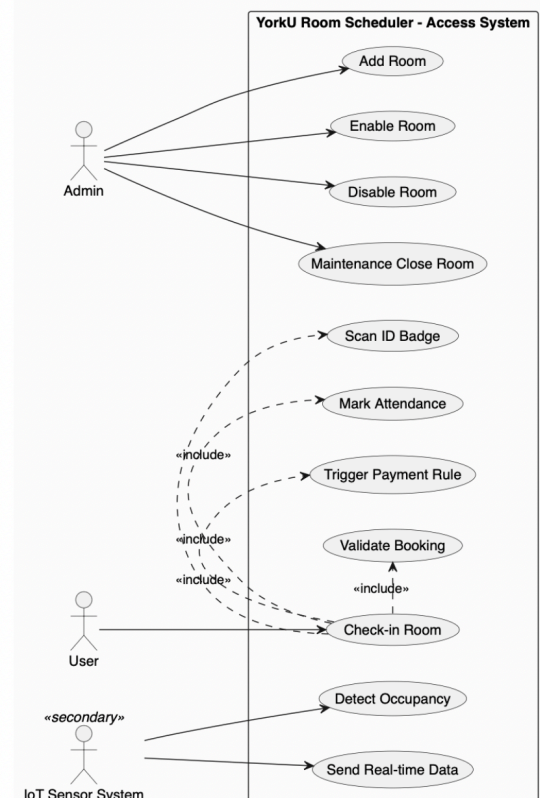
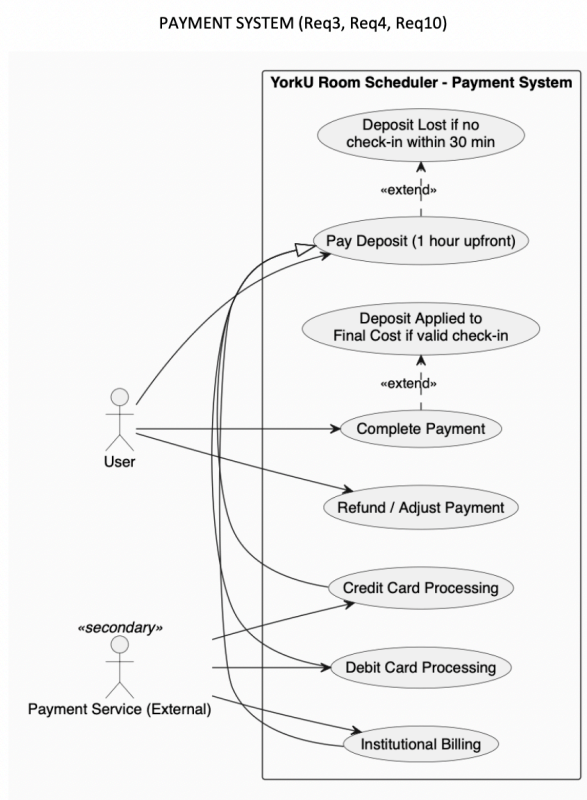
I. Use Case Diagrams



ROOM BOOKING SYSTEM (Req3, Req7, Req8, Req9)



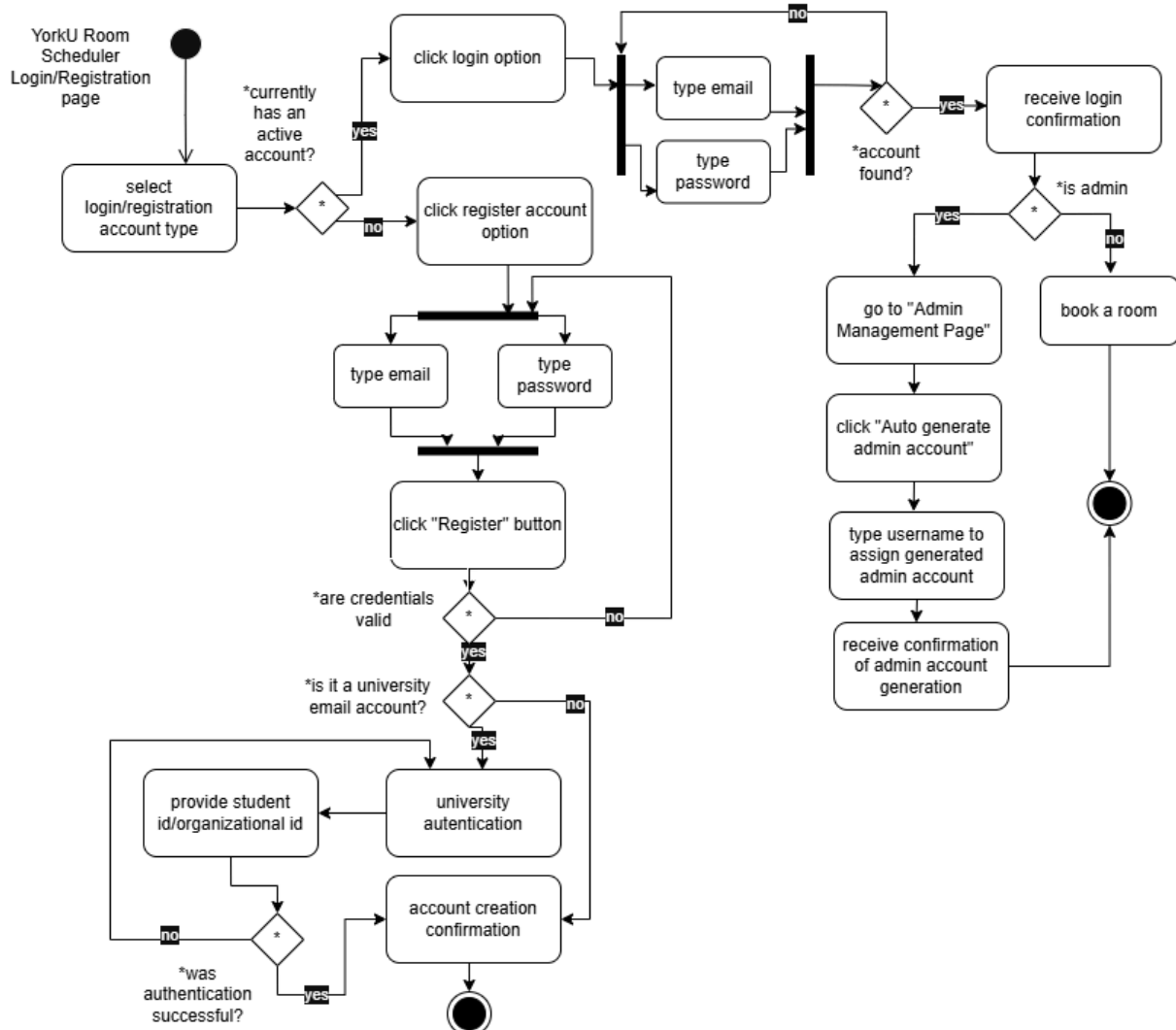
ROOM + SENSOR + CHECK-IN SYSTEM (Req4, Req5, Req6)



- 1) **Account management system:** This diagram models account registration and administration. Users can register and log in, with registration including email validation, password validation, account type assignment, and university verification when required. The Chief Event Coordinator is responsible for generating administrator accounts in accordance with Req1 and Req2.
- 2) **Room Booking system:** This diagram models the room booking workflow. Users can search rooms, view room details, create, edit, cancel, and extend bookings. Booking includes checking room availability, calculating the hourly rate, creating the booking, and requesting the required deposit payment before confirmation, satisfying Req3, Req7, Req8, and Req9.
- 3) **Payment System:** This diagram models payment operations for room reservations. Users can pay the required upfront deposit, complete remaining payments, or request refunds and adjustments. The system also supports multiple payment methods through an external payment service, while enforcing deposit rules based on the user's check in status, satisfying Req3, Req4, and Req10.
- 4) **Room + Sensor + checkin system:** This diagram models room administration and the check in process. Administrators manage room availability, while users check in to reserved rooms. During check in, the system validates the booking, scans the user's ID badge, records attendance, and triggers the appropriate payment rule. Occupancy data is provided by the external IoT sensor system, satisfying Req4, Req5, and Req6

II. Activity Diagram

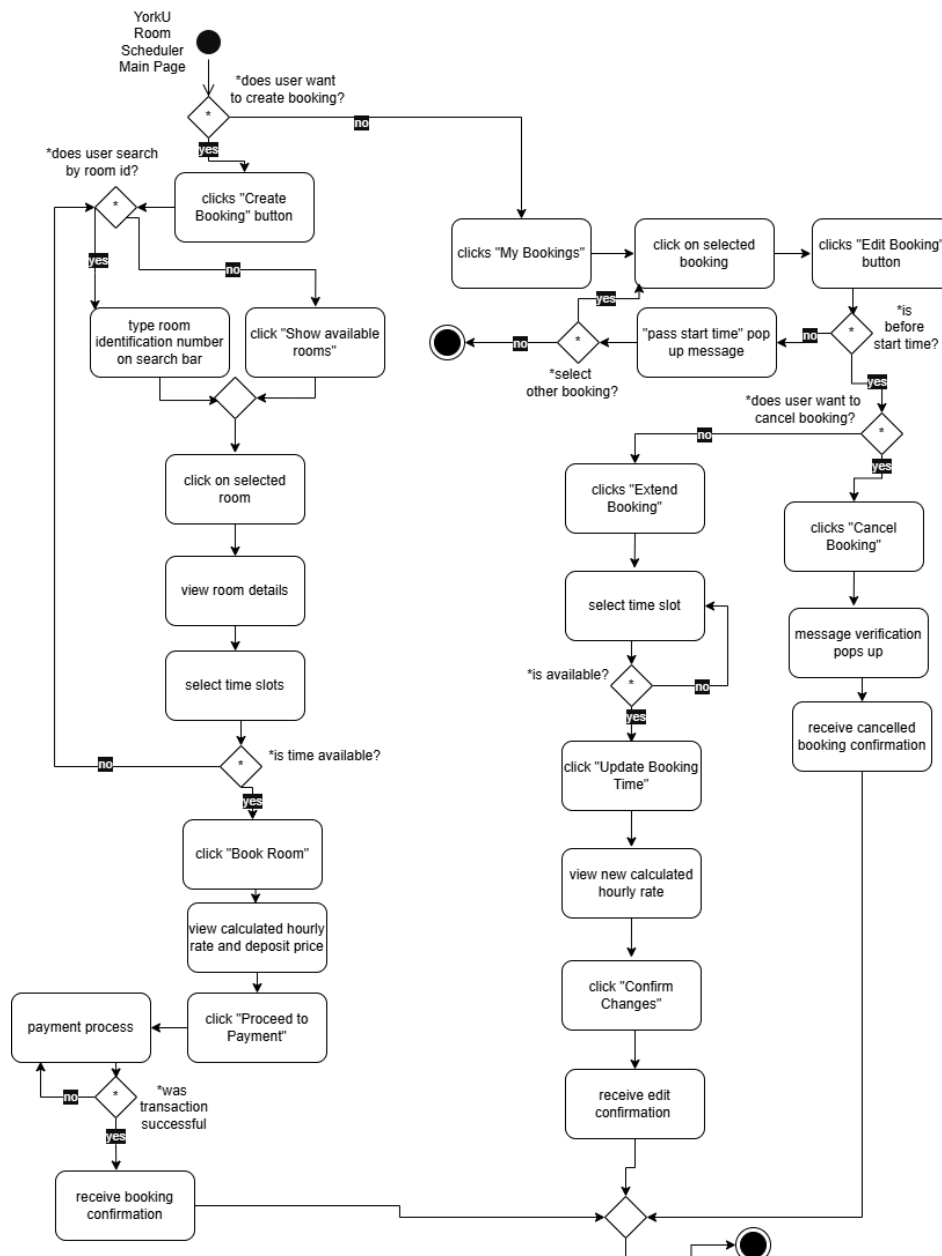
Activity Diagram 1



The landing page architecture is designed around an entry point that handles mutually exclusive system actions based on account presence, initializing by requiring the client to select their target role context to provide an early validation structure. For users accessing pre-existing accounts, the authenticated access control path optimizes user experience and maintains data protection boundaries by utilizing a concurrent UML fork/join block to process email and password credentials simultaneously. Once synchronized via a decision node evaluating database persistence (*account found?), a critical post-authentication design decision evaluates role specialization (*is admin). In strict alignment with system authorization rules, administrative accounts uniquely inherit access to privileged operations like automated account generation, while standard actors bypass these restricted controls entirely and route immediately to the spatial booking subsystems (book a room), cleanly segregating user privileges. Alternatively, the conditional subsystem registration pipeline mitigates faulty account creation by validating input string strengths prior to database modification (*are credentials valid) and evaluating email domain origins to handle institutional constraints. Non-institutional accounts bypass verification and map directly to creation confirmation, whereas domains matching institutional criteria branch into a mandatory verification loop (university authentication) to enforce academic account

verification rules. To capture permanent tracking attributes cleanly without disrupting daily, subsequent login procedures, the collection of identification data (provide student id/organizational id) is encapsulated explicitly within this one-time registration loop, which safely converges into the main lifecycle only upon receiving a successful authentication token (*was authentication successful?), concluding both registration variants at a unified final node termination.

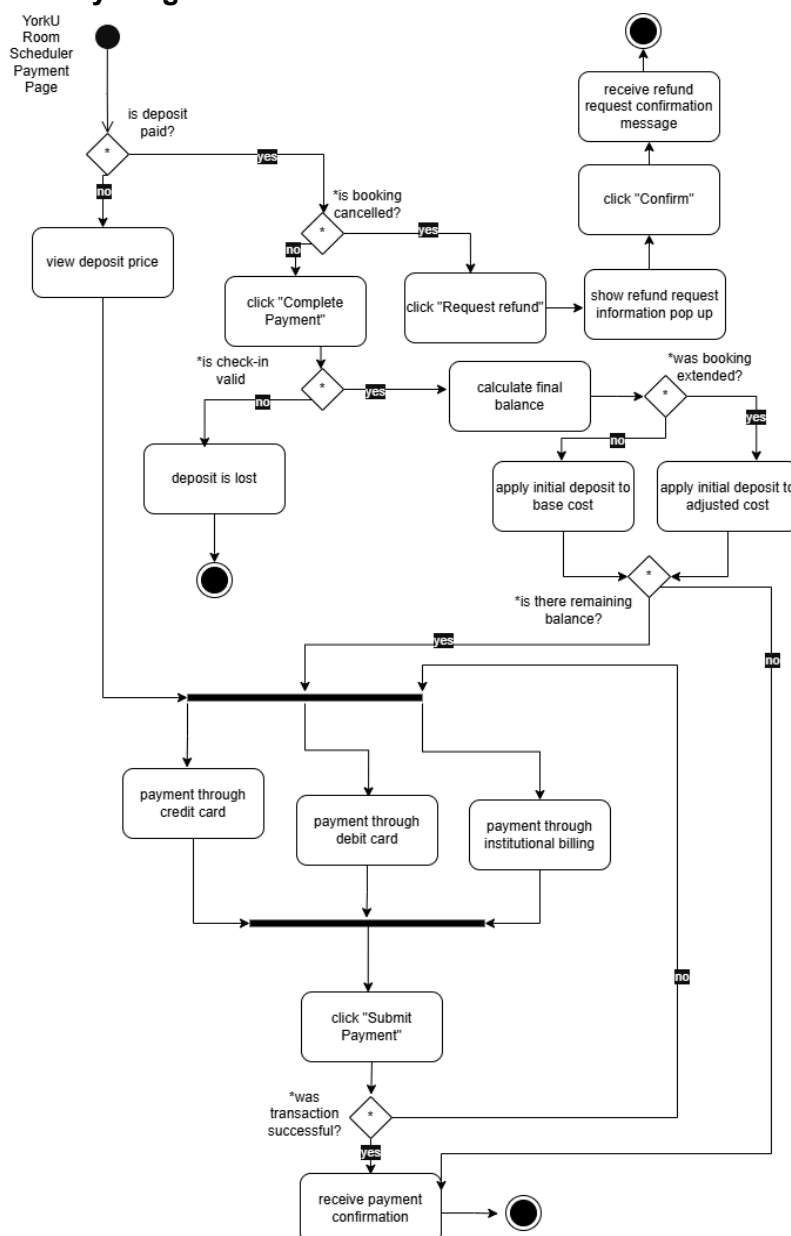
Activity Diagram 2



The architecture of the Room Booking Subsystem maps out an integrated flow for a registered user which clearly separates making a new reservation from changing an existing booking, exactly matching the rules in Requirements 3, 4, 7, 8, and 9. For new reservations, the entry checkpoint explicitly evaluates user intent (*does user want to create booking?), utilizing a decision diamond (*does user search by room id?) to split the control flow into two mutually exclusive behavioral patterns: searching via explicit constraints or pulling a broad system query via click "Show available rooms". Once a space is chosen, the system

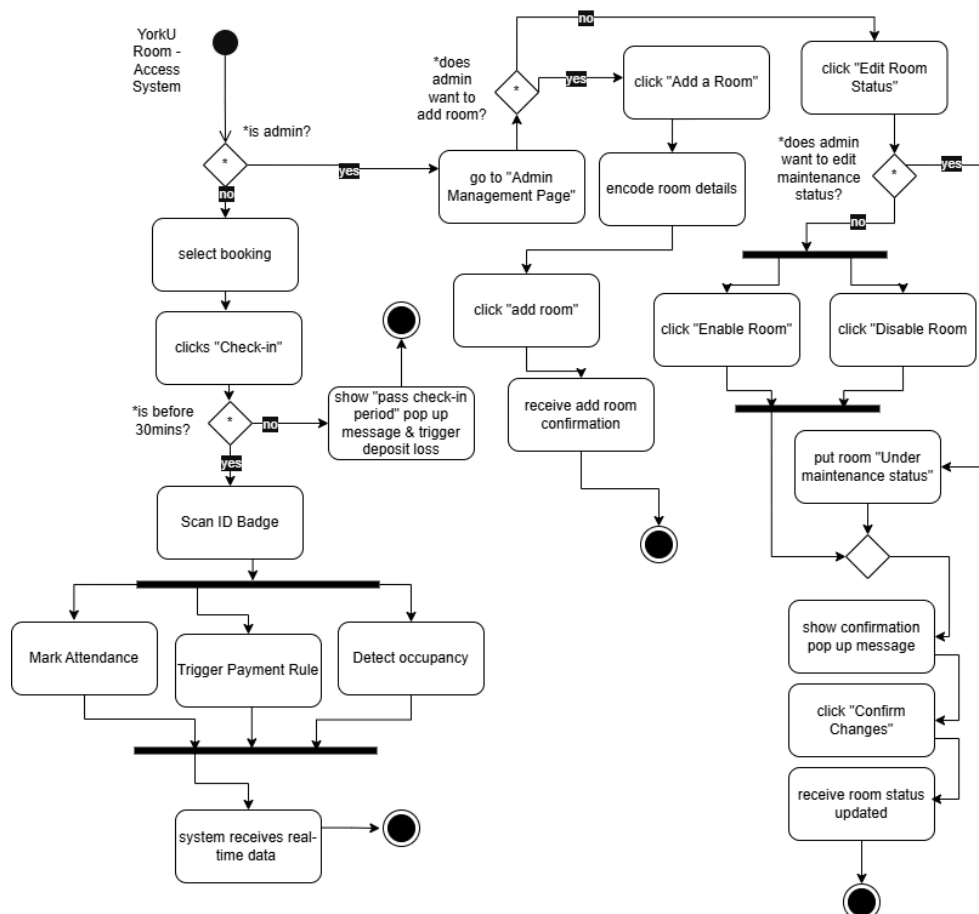
exposes metadata supporting Req7 (view room details and select time slots), and checking real-time scheduling conflicts (*is time available?) after input capture to ensure a clean transaction loop that recycles seamlessly upon failure. Upon validation, the pipeline executes the included use cases Calculate Hourly Rate and Request Deposit Payment via the hourly rate and deposit pricing blocks to enforce the upfront financial criteria of Req4 and Req10. Alternatively, navigating through clicks "My Bookings" routes the actor into an encapsulated management lifecycle where the system immediately enforces the temporal boundary check of Req8 (*is before start time?). If this validation fails, the user is locked out of modifications and routed back to select other bookings; if valid, a choice point separates the workflows, directing cancellation requests through a security validation pathway, or branching into the extension pathway (clicks "Extend Booking") where the system prompts for additional duration and runs a mandatory conflict reevaluation (*is available?) to defend against booking overlaps in accordance with Req9 before both operational paths cleanly converge into a single terminal merge layout.

Activity Diagram 3



The architecture of the Payment Subsystem establishes a comprehensive transactional control flow that handles initial deposit evaluations, room cancellations, check-in validation rules, and final price adjustments to strictly map to Req3, Req4, and Req10. The subsystem initializes via a boundary gateway (is deposit paid?) where a no token enforces the collection of the mandatory 1-hour upfront fee by routing users through view deposit price before entering the processing core, matching the Pay Deposit use case. If a deposit has already been recorded (yes), the system evaluates if the booking is cancelled, directing cancellation requests sequentially through click "Request refund" -> show refund request information pop up -> click "Confirm" to handle the Refund / Adjust Payment use case before hitting a dedicated final node. For active bookings (no to cancellation), clicking Complete Payment triggers a critical check-in verification node (*is check-in valid) ; an invalid check-in immediately forfeits the fee via deposit is lost and terminates the activity, satisfying the 30-minute late check-in penalty. If check-in is valid, the pipeline routes to calculate final balance and checks if the booking was extended, splitting into distinct calculation variants to either apply the initial deposit to the base cost or to the adjusted cost. Following cost reconciliation, a crucial optimization checkpoint (*is there remaining balance?) determines the routing : if no remaining balance exists, the system bypasses external processing gateways entirely to link straight to receive payment confirmation ; if a balance does exist (yes), the flow moves through a choice node supporting the three mutually exclusive processing routes of Req10 (Credit Card, Debit Card, and Institutional Billing) before funneling into a final transaction success gateway that recycles failures or terminates successfully upon confirmation.

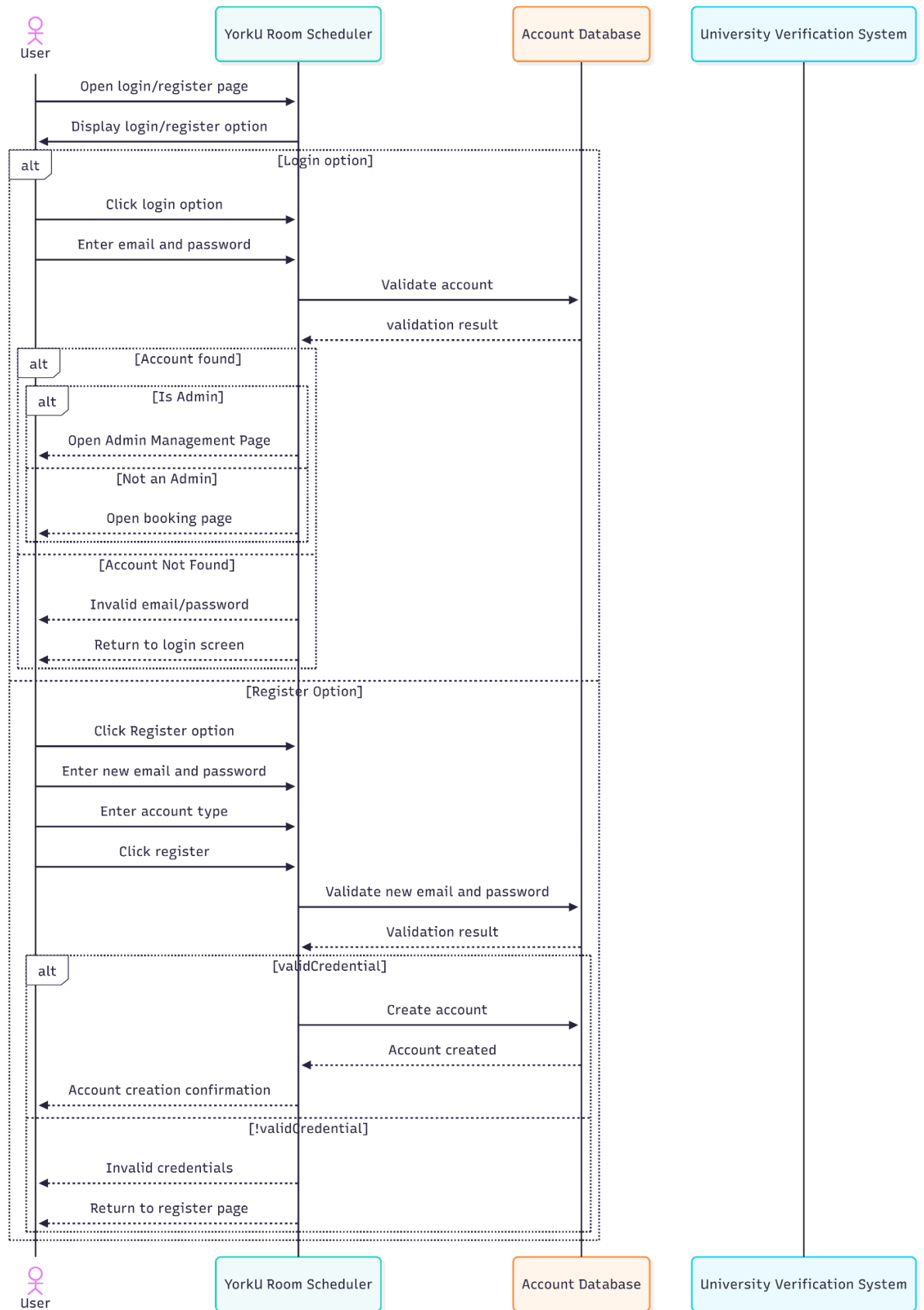
Activity Diagram 4



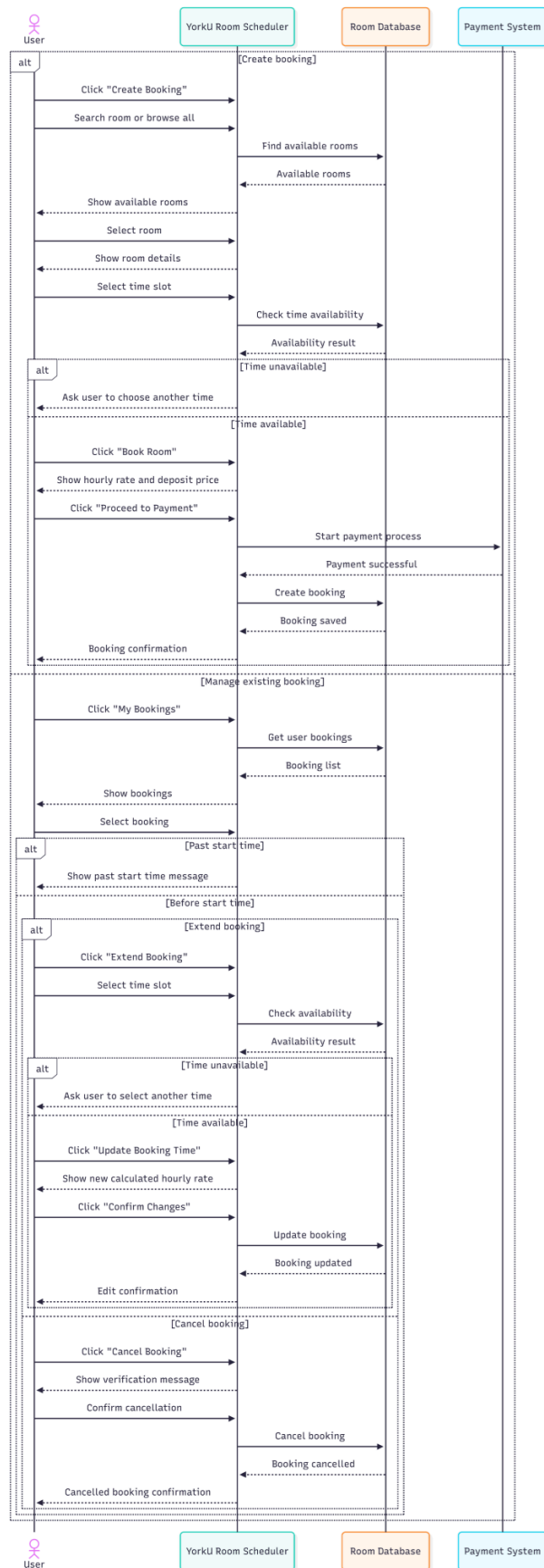
The architecture of the Room Access & Sensor Subsystem shows physical environment interactions, automated hardware events, and administrative configurations by coordinating real-time field hardware with backend authorization rules to satisfy Req4, Req5, and Req6. The workflow initializes with a role-based access control gateway (*is admin?) that forks system capabilities based on structural privilege levels : routing authorized personnel (yes) to go to "Admin Management Page" block to handle spatial configurations , while redirecting standard occupants (no) through a strict validation sequence. To satisfy the check-in penalty policies of Req4, a decision diamond (*is before 30mins?) captures entry times relative to the scheduled lease, routing late arrivals (no) to trigger deposit loss and immediate session termination , whereas users arriving within the window (yes) interact with physical door peripherals (Scan ID Badge). This action activates a solid black UML fork bar that splits control into three concurrent, asynchronous operational tracks: logging events via Mark Attendance, processing financial rules via Trigger Payment Rule, and querying field hardware via Detect occupancy (Req5). Then converging at a matching UML join bar to push a synchronized data stream through a final logging block (system receives real-time data). Alternatively, the administrative pipeline isolates actions to manage room availability states in alignment with Req6, utilizing a choice node (*does admin want to add room?) to route inventory provisioning (yes) through explicit encoding and confirmation states , or status modifications (no) through an operational maintenance decision split. This choice point either transitions rooms out of active circulation via put room "Under maintenance status" or navigates a nested selection block to execute mutually exclusive availability toggles (click "Enable Room" or click "Disable Room") , forcing all three status branches to converge at a centralized merge diamond to funnel through a unified database validation pattern (show confirmation pop up message -> click "Confirm Changes" -> receive room status updated) before hitting a clean final node termination.

III. Sequence Diagrams

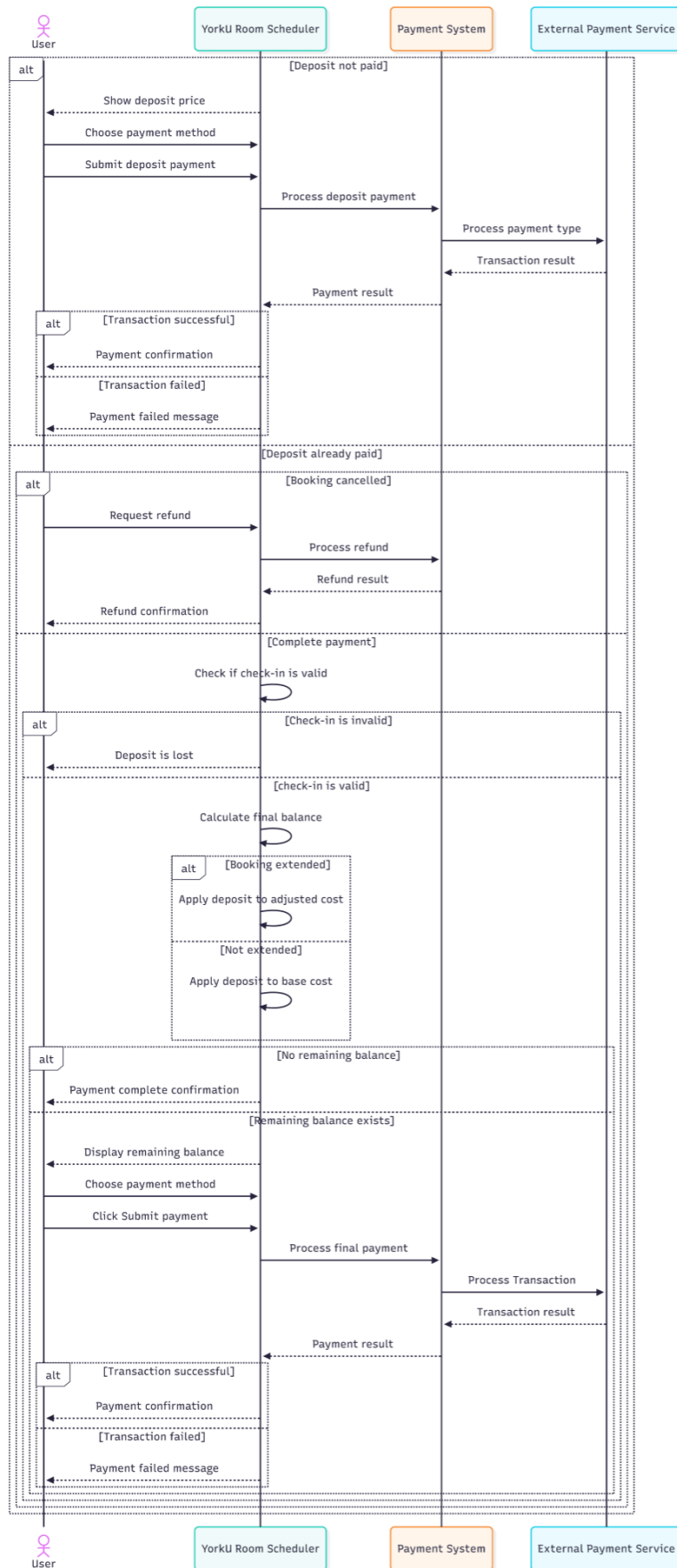
Sequence Diagram 1 (Login/Registration)



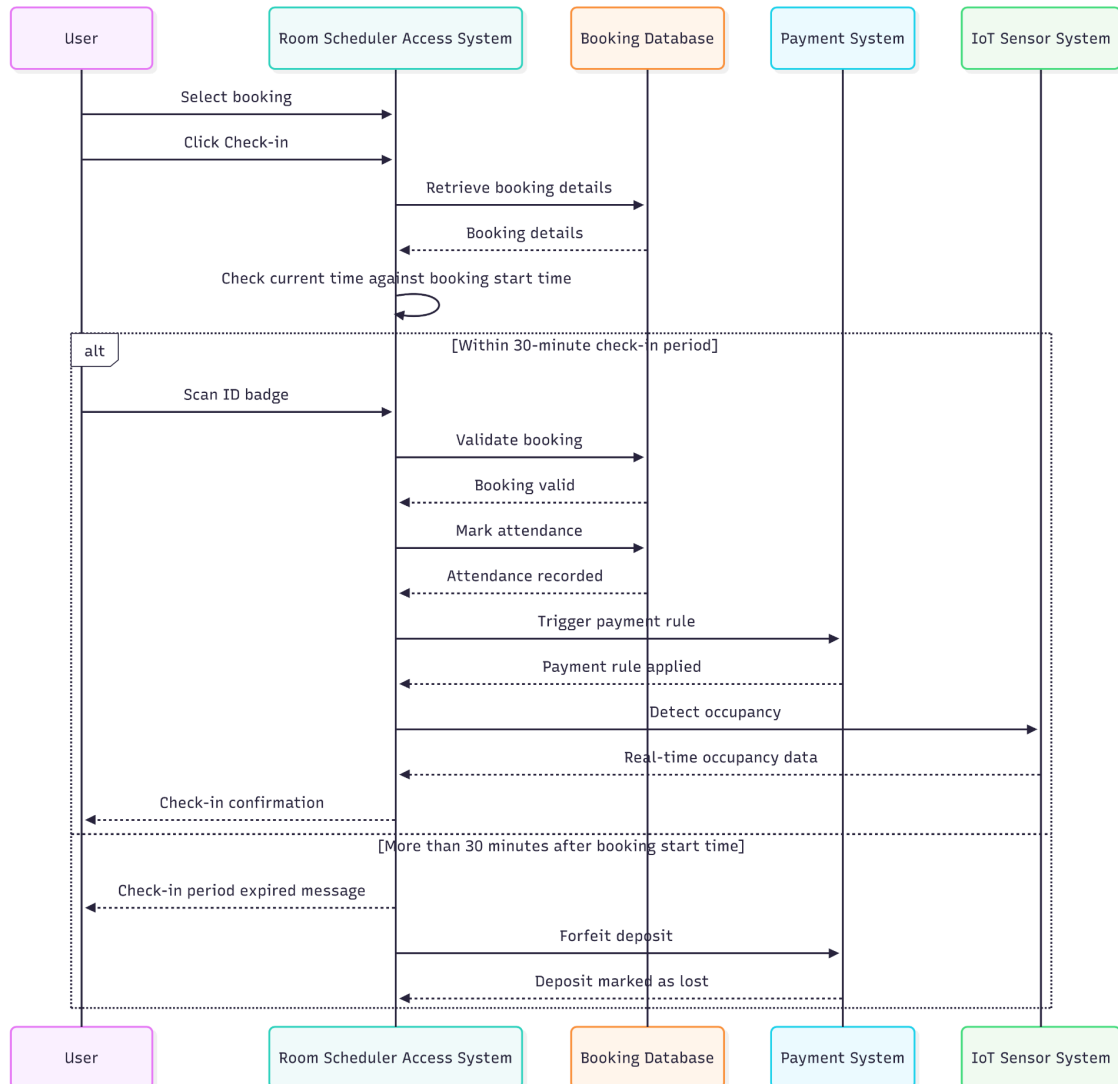
Sequence Diagram 2 (Booking)



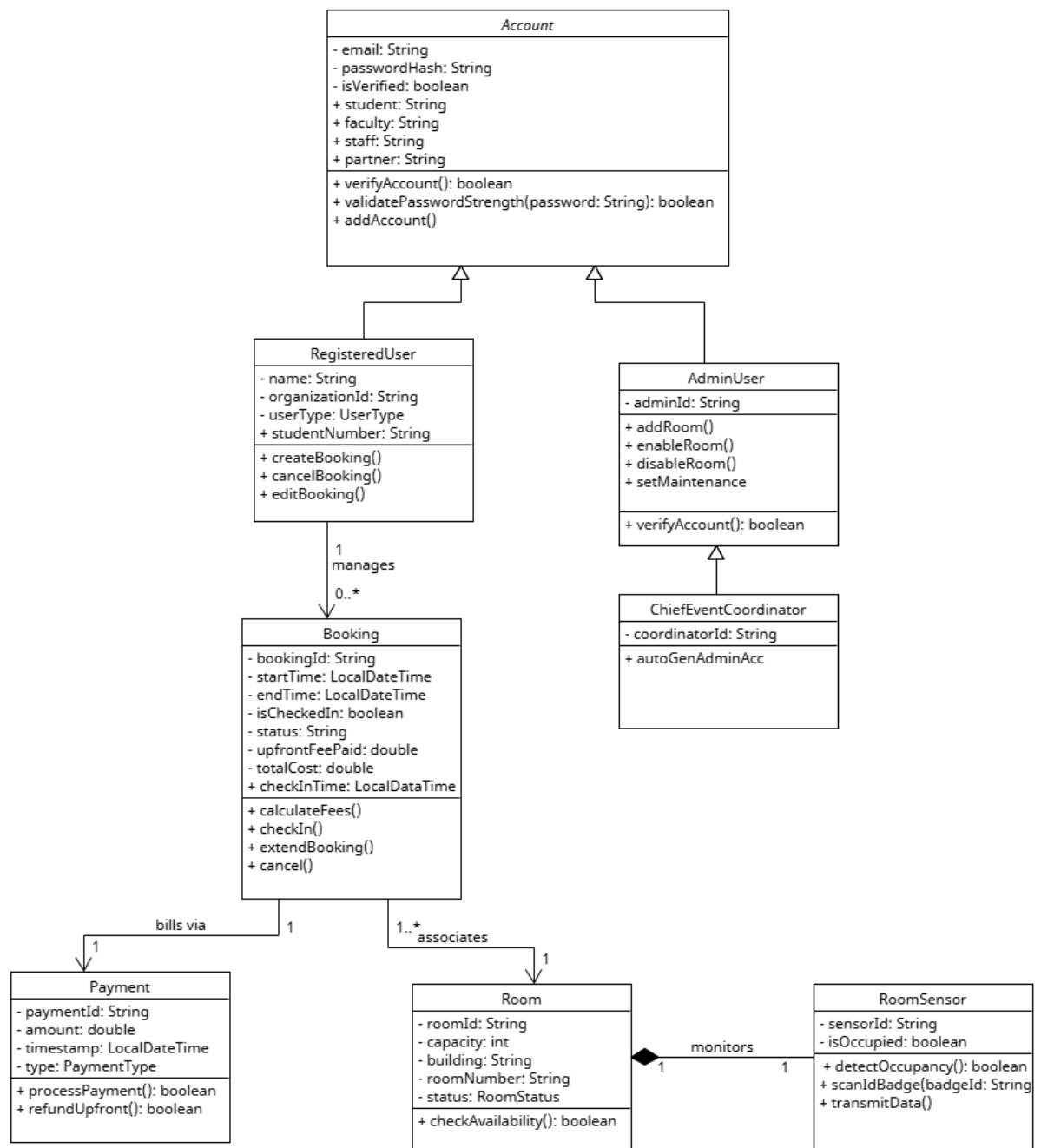
Sequence Diagram 3 (Payment)



Sequence Diagram 3 (CheckIn)



Part II: Design



The above UML class diagram depicts the University Room Scheduling and Automated Access System by making structural connections between user accounts, bookings, infrastructure rooms and payments. This architecture follows a hierarchical identity management system in which the general Account (with attributes email, passwordHash, and isVerified) splits up into two subclasses as AdminUser (which contains the unique class ChiefEventCoordinator that has a unique method autoGenAdminAcc()) and RegisteredUser that connects each profile to their booking history based on organizationId and studentNumber. A

RegisteredUser manages multiple Booking classes with cardinality (1 to 0..*) over time in which each Booking manages the scheduling and finance (startTime, endTime, checkInTime, upfrontFeePaid, totalCost) of exactly 1 physical Room. For automated and real-time access control, each Room (having spatial information like capacity, building, and roomNumber within a Room_Status) makes a strong connection (1 to 1) with RoomSensor that keeps track of the Room through its isOccupied status and badge scanning. Lastly, each Booking maintains a relationship of cardinality (1 to 1) with Payment to securely process transaction amount using PaymentType.

Part III: Report on AI-Assistant Adaptation

Task 1: Explain the Role of the AI Assistant

We incorporated AI into various parts of the project depending on the tasks that we were assigned to help correct our mistakes and think in the right direction. None of us went with exactly what AI told us and copy pasted that output and then later submitted it. We tried to do the tasks ourselves initially and asked AI for feedback to proofread or double-check our work if it was in accordance with the instructions provided. Below is a prompt-by-prompt breakdown of our conversations and how we utilized the output given. Starting with the diagrams we did use Gemini, ChatGPT and other tools to help us identify our gaps in understanding the project requirements For Example: We drew a diagram that we thought was right and fulfilled all the conditions outlined in the deliverable and then we uploaded that diagram to an AI tool and asked it to verify if we did it correctly. Our goal was to use AI to prompt us in the right direction so as to fulfill all requirements. For the report we did use AI to identify what structure we had to follow as it was unclear from the deliverable instructions. We wrote most of the text inside using our own thinking, inferences and notes. However, we did use AI to polish our report to help fix grammatical and other mistakes.

Use Case Diagram

AI Tool Used: ChatGPT (GPT-4o)

The use case diagrams were built in draw.io from scratch. Once a draft was done, screenshots got uploaded to ChatGPT along with a prompt roughly like:

"Given this UML diagram image, if I was to design a [payment system / account system / etc.] for York Uni, would this model work for [scenario]? Do not update the diagram . just check if it works without logical error and make sure the UML diagram is syntax correct."

The "don't update it" part of that prompt was important. We'd seen what happens when you just ask AI to "improve" a diagram. it rewrites everything according to its own preferences and you lose what you were actually trying to model. We wanted it to poke holes, not take over. One round per diagram, then another after the team had reviewed things.

What it found:

On the payment use case diagram, ChatGPT caught two things we had missed:

1. The «extend» arrows were solid lines. They're supposed to be dashed. We'd been so focused on whether the diagram made logical sense that nobody had looked twice at the line styles.
2. Debit card was nowhere in the diagram. Req10 lists credit cards, debit cards, and institutional billing. We had the other two, just not debit. Nobody had noticed.

What we took from it:

The debit card gap was the one that actually mattered, that would've been a direct requirements miss. The dashed arrow thing was more of a notation fix. Honestly though, ChatGPT buried these two things in a lot of output about things we had done correctly, so it took some reading to find the actual problems. Skimming for the words "missing" or "incorrect" ended up being more efficient than reading it top to bottom.

Activity Diagrams

AI Tool Used: Google Gemini

Gemini got used the most out of any tool in this project. All four activity diagrams were drafted in draw.io first, then Gemini was brought in for feedback through an ongoing conversation, screenshot by screenshot, question by question. The Gemini link above has the full exchange. Here's what actually happened in each key prompt.

Prompt 1 — Login/Registration diagram, fork/join issue:

"I'm creating an activity diagram based on the use case diagram. For the creating registered account I need some help on the design decisions. Currently I have the type of email and password as a fork (so they are in parallel)..."

The reasoning behind the fork was that on a real web form, you type both your email and password before clicking anything. Neither one has to come first. Gemini actually agreed with that logic. Where things went wrong was in how the diagram itself was drawn. Three problems: there were two solid black join bars sitting on top of each other when you only need one, the "yes" arrow out of the validation decision diamond was going nowhere. It was just floating and the "no" path for failed validation was looping back to the fork bar, which you can't do in UML. It needed to loop back to the action states themselves.

The stacked bars were just a tool glitch. draw.io had duplicated something and it looked fine at normal zoom. The floating arrow was harder to catch because from far out the diagram looks connected even when it isn't. Zooming in was basically the fix.

Prompt 2 — Can you have two activity final nodes?

"I'm not sure if I can have two activity final nodes."

There were two separate execution paths in the same diagram. one for login, one for registration and both needed an ending. Having two of those filled black dots just looked wrong, wasn't sure if UML even allowed it. Gemini said yes, you can have multiple. Once any token hits one, the whole activity stops. And actually having a separate endpoint per path is cleaner than drawing long crossing lines down to one shared dot at the bottom.

That answered the question. Then Gemini also threw in a whole second list of things still wrong with the diagram: the double join bar from Prompt 1 was still there, the "no" branch on the university email check was stuck in an infinite loop because it kept bouncing back into the confirmation block instead of going anywhere, and the "yes" branch on the authentication

check was skipping past the confirmation block entirely. None of this was what we asked. Had to re-read the response a couple of times to figure out what was new feedback vs. what we'd already fixed.

Prompt 3 — Course slide for the join bar:

"For the join bar this is what my course has on it."

A screenshot of the lecture slide on Fork and Join Nodes got uploaded alongside this prompt. The reason was pretty practical. Gemini had explained the join bar fix but in a generic UML way, and we wanted to make sure our diagram matched specifically what the course expected, not just what's technically valid in UML generally.

Gemini worked from the slide directly. It looked at the slide's example (one fork, two concurrent actions, one join bar) and said: your diagram should look exactly like that. Fork into "type email" and "type password," single join bar, then "click Register button." Delete the extra bar. That was it. Giving Gemini the course's own notation to reference instead of just asking for "the right way" made the answer actually useful for this specific submission.

Prompt 4 — Does the admin account flow make sense?

"Is the creation of admin account logically make sense?"

So the admin account generation flow had been drawn right after the regular user's login confirmation. Seemed fine at the time. But what that actually says in UML is: every person who successfully logs in immediately gets checked to see if they're the Chief Event Coordinator, and if they're not, the session terminates. Meaning regular users would log in and get kicked out. That's obviously not right.

Gemini flagged this and gave two ways to deal with it. One option was adding a "Select Action" decision diamond after login. One branch going to room booking for regular users, another going to the Admin Management Page for the Chief Event Coordinator. The other option was scrapping the admin flow from this diagram entirely and making a separate one that starts with the coordinator already logged in. We went with the first option. Keeping it in one diagram made the role separation more visible, and the branching logic made more sense architecturally.

Prompt 5 — "Show available rooms" felt redundant:

"The show available rooms does not make sense when im going to ask again later if its available."

Looking at the booking diagram, showing available rooms early and then checking availability again later genuinely seemed like the diagram was doing the same thing twice. Gemini explained the difference: "show available rooms" is filtering out rooms that are closed or under maintenance, it's reducing the list. The later check is asking whether this specific room is free during the exact time slot you selected. So they're checking two different things, not the same thing.

But the bigger issue Gemini pointed to was the Fork Bar we'd used for the search method, which was one branch for searching by room ID, one for browsing all rooms. A fork bar means both happen simultaneously. Nobody is searching by ID and browsing everything at the same time. That needed to be a Decision Diamond. Swapping the fork/join bars for a decision and merge diamond pair took care of the redundancy problem at the same time.

Prompt 6 — Payment diagram and the payment method fork:

"Does the activity diagram reflect the use case diagram (an edit will be made that separates the refund and adjustment price)?"

The payment method section had a Fork Bar splitting into credit card, debit card, and institutional billing. In UML, fork bars implies concurrent meaning all paths fire at once. So the diagram was saying users pay with all three methods simultaneously. A Decision Diamond was what it needed instead since the user picks one method, not all of them.

Beyond that, Gemini mapped out how the deposit-to-balance calculation should flow: calculate final balance first, then check whether the booking was extended, apply the deposit to either the base or adjusted cost, then check if there's any remaining balance. If there isn't, the user is done, no card needed. If there is, then go to payment method selection. We used this structure in the final diagram because it solved a problem we hadn't thought about: the original version would've made users re-enter payment info even when their deposit already covered the total.

Prompt 7 — Access/Room Management diagram, full requirements check:

"Req1: Anyone can create an account using a unique, valid email and a strong password..." (All ten requirements were pasted in alongside the diagram.)

For this one we pasted in all ten requirements so Gemini could check each one directly against what the diagram showed. The main thing it confirmed was that the fork/join bars for Mark Attendance, Trigger Payment Rule, and Detect Occupancy were actually correct here. When a sensor scans a badge, all three of those things happen at the same time. That's genuinely concurrent, not a user choice. Compared to the payment method fork earlier which was wrong, this was a good clarification of when you actually should use fork/join vs. when you shouldn't.

It also caught a routing bug we'd missed: the Enable Room branch wasn't feeding into the merge diamond, it was jumping straight to the confirmation pop-up, skipping the merge entirely. And several decision diamonds were missing guard labels in square brackets. Missing those isn't catastrophic but it's sloppy UML, and it would've been easy marks lost.

Sequence Diagrams

AI Tool Used: AI for syntax reference; mermaid.ai for rendering

The sequence diagrams were written in Mermaid rather than drawn visually. The reason was just iteration speed. Changing a word in a text file and re-rendering is faster than moving boxes around in draw.io. Learning the syntax from scratch took a bit. AI got used mainly as a

faster way to look things up: how do you open an alt block, how do you add guards in square brackets, what does a self-call look like when a system is doing an internal calculation.

The outputs were mostly short code examples, not explanations. Things like a snippet showing how to declare a participant, or the exact syntax for nested alt blocks. The design side of things like which participants to include, what the messages should be, how to handle the check-in timing and payment outcomes — that came from the activity and use case diagrams. AI was basically being used as a documentation shortcut, not a design tool.

Class Diagram

AI Tool Used: Google Gemini

Two Gemini sessions were used for the class diagram. The first checked against the requirements. The second checked the relationships against the activity diagrams.

Prompt 1 — Requirements check:

"Hello, Is this UML diagram in accordance with these reqs: [Req1–Req10 pasted in full]"

Gemini went through each requirement and said what matched and what didn't. Most of it was fine — RoomSensor, AdminUser, Payment, Booking were all solid. Three things needed fixing:

RegisteredUser had an attribute userType: UserType was never actually defined anywhere in the diagram. The type was referenced but never declared, so the attribute pointed at nothing, Gemini's suggested adding an <<enumeration>> UserType block. We chose not to introduce a formal enumeration, since enumeration isn't part of the notation we used so far in the course. The four account types are represented as plain attributes on the Account class (student, faculty, staff, partner), with userType kept on RegisteredUser as a simple field rather than a declared enum. Related to that, calculateFees() in the Booking class had no clear way of knowing which hourly rate to apply to which user type.

The other issue was multiplicity on the inheritance arrows. The lines from RegisteredUser and AdminUser pointing up to Account had numbers on them — 1, 0..*. That's not valid. Inheritance lines in UML don't get multiplicities. A child class is just an instance of the parent. There's nothing to quantify. It looked fine at a glance because the numbers blended into the diagram, but Gemini flagged it and it had to be removed.

Prompt 2 — Relationship check against activity diagrams:

"Does this diagram represent the relationship between the classes as per the reqs 1–10 and this activity diagram?" (Activity diagrams were uploaded alongside the class diagram.)

The multiplicity between Booking and Room was wrong. The diagram had 1..* on the Booking side, which means a Room cannot exist without at least one booking already

associated with it. But an Admin can add a room that nobody has booked yet, that's literally what Req6 says and it's shown in Activity Diagram 1. The corrected value is 0..* on the Booking side, since a room can exist before any booking. The diagram currently shows 1..*.

Gemini also flagged that AdminUser was missing a method for the "Edit Room Status" flow from Activity Diagram 1. Something like setMaintenanceRoom(). As noted above, the account types are represented as plain attributes on Account rather than formal UserType enumeration.

One suggestion we didn't use: Gemini recommended adding a separate Rate entity or configuration class to explicitly model the hourly pricing logic. That felt like more than what's needed at this stage. The calculateFees() method already communicates that the fee calculation happens at the Booking level, and adding a whole separate class for it seemed like overengineering for Deliverable 1. Might revisit for Deliverable 2 if design patterns push us in that direction.

Task 2: Arguments and Resolutions

Not every task had a real disagreement. Some of the feedback was just plain fact. A missing requirement, a wrong line style, and we accepted those without arguing because there was nothing to argue about. The cases worth writing up are the ones where the AI's logic was wrong for our system, or where it was pushing us somewhere this project did not need to go. We go through it on a task by task basis.

Use Case Diagrams

The two things ChatGPT caught on the payment diagram were the missing debit card and the solid <<extend>> arrows that should have been dashed. Neither was a judgment call. Req10 names debit cards. <<extend>> is dashed by definition so we just fixed them and moved on.

One actual disagreement was with how the AI behaved rather than any specific suggestion. Early on we asked it to 'improve' a diagram, and it wrote the whole thing into its own preferences, so we had to figure out how to prompt the AI to not do so. That wasn't what we wanted, the point was to find holes, not hand the design over. We fixed it by locking the prompt down "do not update the diagram, just check if it works without logical error." That kept the AI in its "reviewer" role.

Activity Diagrams

Most of our back and forth happened here, since Gemini got used the most on this part.

The disagreement was on the 2nd Activity Diagram. We thought "show available rooms" and the later "is time available?" check were redundant. It looked like the diagram was asking the same thing twice. Our instinct was to cut one. So we put that into Gemini expecting it to agree, but it didn't. It argued that the two are different operations. "Show available rooms" filters out rooms that are closed or under maintenance, which is a list level filter. The later check asked whether one specific room is free for the exact slot you picked, which is a slot

level check. We worked through it and agreed it was actually right. They aren't the same question so we didn't delete anything. Instead we fixed the real problem, which was the fork bar on the search method implying that searching by ID and browsing all rooms happen at once. That became a decision/merge pair, and the redundancy feeling went away on its own. We lost the surface argument but still came out with a better diagram.

The admin flow on the 1st Activity Diagram was different. Here we agreed with the AI on the problem. Drawing admin generation right after login meant every regular user would get role-checked and dropped at a final node. But the fix was left to us. Gemini gave two options: one kept everything in a single diagram behind a "Select Action" decision branch and the other split the admin path off into its own diagram. We picked the first since keeping both roles in one view made the split between a regular user and coordinator easier to see.

Sequence Diagrams

No design disagreements here. The AI was only used as a syntax reference for Mermaid. How to open an alt block, how to write guards, what a self call would look like. Nothing about the actual design. There was nothing to argue about because we weren't asking for a design input. The structure came from the activity and use case diagrams we'd already locked down.

Class Diagrams

This was the strongest disagreement in the project. Gemini recommended adding a separate Rate entity, or a config class, to model the hourly pricing on its own. We said no to this because the requirements spell out that design patterns aren't required yet and that we will brush up against it in one of the next deliverables. Pulling pricing into its own class is exactly that kind of later move. `calculateFees()` already sits on Booking and shows where the fee calculation happens. Wrapping a whole class around it would have been over engineering for what this project asks. So we rejected it and flagged it. This was the AI reaching for a "more structure is better" when the scope did not call for it.

Note

One pattern showed up across all of these. The AI wasn't usually wrong on the facts. On UML rules it was almost always right. The problem with the AI was that it was over-produced. It buried the two real fixes on the payment use case diagram inside paragraphs of praise for things we'd already gotten right. At one point it re-listed problems we had already fixed as if they were new. We had to read its responses critically, almost like reviewers. That habit of over-explaining was the thing we argued with most, more than a single diagram component.

Task 3: Verifying AI Responses

3.1 Purpose of Verification

The AI assistance we used to produce UML was both fluent and confident, but it doesn't make it correct. The output often looked legitimate but it quietly violated UML semantics, misread requirements, or invented logic the requirements never asked for. The deliverable

itself says a real world developer should be able to build the system from these diagrams, so we couldn't take anything on trust. Every AI suggestion got checked against three rules before it went near our diagrams.

1. The projects 10 requirements
2. The notation taught in the course
3. Behaviour we have committed to the previous diagrams

The point of verification wasn't to confirm that a response sounded right. It was to confirm each suggestion conformed to the UML standards we'd been taught and to our own system, and to catch the place where it didn't.

3.2 Verification Process

We verified all of the AI responses by using an approach that had a list of requirements for the AI to be cross examined:

1. Requirements check: Every suggestion got mapped back to the specific requirement it claimed to satisfy. If the AI proposed something no requirement called for, or skipped one a requirement demanded, that was a flag. This is how the debit card gap got caught, Req10 listed credit, debit, and institutional billing, and the payment diagram only had two of the three. Same with the hour rates, when Gemini said `calculateFees()` had no way of knowing which rate to apply, so we checked that against Req3 instead of just trusting that something was missing.
2. Course standard check: Where a suggestion was about notation specifically, we checked it against what the course taught. Some things had to be researched with the help from AI, and the two didn't always line up. We used the course slides as the reference and treated the AI as a fact checker.
3. Cross diagram check: The class diagram has to support the behaviour in the activity diagrams, we checked suggestions across diagrams instead of in isolation. The Booking-Room multiplicity is the clearest example. It looked fine on its own, but lined up against Activity diagram 4, where an admin adds a room nobody booked yet, the `1..*` on the Booking side is wrong, because it would mean a room cannot exist without a booking. Comparing both diagrams was the only way to catch this. The corrected value is `0..*`, not the `1..*` shown.
4. Reasoning check: We looked at why the AI said something. It gives right answers but that doesn't mean it's the correct answer for you, a correct rule but in the wrong place. Fork/join was the case where this mattered most. Gemini told us to replace fork bars with decision diamond for the payment methods, which was correct, since the user picks one method. But that same fork was right for the sensor flow. Mark Attendance, Trigger Payment Rule, and Detect Occupancy all fire at the same time when a badge is scanned. If we just accepted this rule we would have a broken diagram.

Only after a suggestion cleared the checks, it then was applied to a diagram.

3.3 Example of an AI Suggestion That Needed Verification

When we evaluated the class diagram, the assistant we used (Google Gemini) flagged the generalization arrows running from RegisteredUser and AdminUser up to Account. Its response was:

“In standard UML, Generalization (inheritance) lines should not have multiplicities (like 1 or 0..*) because a child class is an instance of the parent class. Remove any numbers or text sitting on those inheritance arrow shafts.”

This had to go through the process of verification because it is a UML standard claim and not a requirements claim. This was AI inserting a rule about notation itself, which we then checked against some of the course material on generalization and association and the claim is correct. This is because generalization has an ‘is-a’ relationship and not structural association. Multiplicity also belongs only on associations, aggregations, and compositions so the AI’s reasoning was correct and the suggestion passed our checks.

3.4 Outcome

Accepted and applied. The multiplicity numbers came off the inheritance which left an open triangle generalization arrow from RegisteredUser and AdminUser to Account, and that corrected version is what’s in the final class diagram. It looked fine at first which is why checking notation through a check was worth doing.

Not everything was landed cleanly. In the same class diagram session, Gemini recommended defining a UserType enumeration to back the userType attribute and hold the four account types. The conclusion wasn’t wrong in pure UML terms, but it failed our course standard check, enumerations aren’t a notation that are taught in any of our slides. So we modified the approach instead of taking the suggestion as is. We represented the account types as plain attributes on the Account class rather than introducing an enum. A plain attribute also fits Req1 better than a fixed enum would, since a new type is just a new value with no structural change.

Task 4: Application of AI-Suggested Content

4.1 Original Prompt Used

While building the payment activity diagram, the deposit logic was unclear: the diagram just routed “deposit applied to final cost” step directly into the card payment block which in turn forced a user to enter payment details even when the deposit covers the balance. The assistant was asked whether the diagram aligned with the use case with the full list of Req1-Req10 given as context. The prompt sent was:

“Does the activity diagram reflect the use case diagram (an edit will be made that separates the refund and adjustment price)”

4.2 Exact AI Response Applied

The assistant returned the following, which was then applied to the diagram:

"User clicks Complete Payment. Decision: Is check-in valid? No → deposit is lost → Activity Final Node. Yes → Move to a calculation action block: calculate final balance. Decision: Was booking extended? No → apply initial deposit to base cost. Yes → apply initial deposit to adjusted cost. Both paths merge into a single decision diamond: Is there a remaining balance? No → Point directly to receive payment confirmation. Yes → Route down into your payment selection diamond (Credit / Debit / Institutional)."

4.3 Where This Appears in the Final Submission

This appears in the third activity diagram (Payment). After "Complete payment" the diagram branches on check in validity (invalid -> "deposit is lost" -> final node), then runs "calculate final balance" branches on "was booking extended?" into "apply initial deposit to base cost" / "apply initial deposit to adjusted cost," merges into "is there remaining balance?" decision, and from there either goes straight to "receive payment confirmation" (no balance) or down into the "choose payment method" selection (credit / debit / institutional billing) before "Submit Payment." The "is there remaining balance?" check and the bypass straight to confirmations are taken directly from the AI response.

4.4 Rationale for Accepting the AI Output

The output was accepted with little modification for three reasons. First it fixed a genuine logic error we had which was that without the remaining balance checkpoint, the diagram forced every user (including one whose deposit fully covered the cost) to reenter their payment details for a \$0 charge. Second, it implemented Req4 properly, the one hour deposit is applied to the final cost on a valid check in, which is exactly what the branch on check-in validity steps encodes. Third, it even cleanly separated the standard case from the extended booking case (Req9) by splitting the base cost from the adjusted cost, which matched the edit we had already planned and aligned with the extension points in the payment use case. Because the suggestion was both UML valid and stayed faithful to the requirement, it did not need significant rework.